Technische Universität Hamburg

Data Science Foundations

Low-dimensional Control under Embodiment Constraints: From LQR to Reinforcement Learning

Bachelor's Thesis
am Institut Data Science Foundations
Technische Universität Hamburg

vorgelegt von
Alexander Kranz
am
08.09.2026

Erstprüfer:	Prof. Dr. N. Ay
Zweitprüfer	Dr. Manfred Eppe
Betreuung:	Dr. Adwait Datar

Alexander Kranz
geboren am: 09.02.2004
Matrikelnummer: 590156
Studiengang: Data Science

Abstract

Your English abstract here (mandatory if written in English and recommended otherwise).

Zusammenfassung

Hier die deutsche Zusammenfassung einfügen (notwendig).

Contents

1 Introduction **1**

2 Preliminaries **3**
2.1 Linear State-Space Models 3
2.1.1 Discretization 4
2.2 The Double Integrator 5
2.3 Characteristics of a system 6
2.3.1 Stability 6
2.3.2 Controllability 7
2.3.3 Observability 8
2.4 The Linear Quadratic Regulator 9
2.4.1 The optimal gain 10
2.4.2 The cost as a function of the gain 10
2.5 Uncertainty and Disturbance Models 11
2.5.1 Measurement Noise 12
2.5.2 Control Noise 12
2.5.3 Parametric Uncertainty 12
2.5.4 External Disturbance 13
2.5.5 The full loop 13
2.6 Robustness Metrics 14
2.7 Notation 14

3 Background **15**
3.1 Motivation 15
3.2 Approach 15

4 Robustness of LQR with low-dimensional Input **17**
4.1 Setting 17
4.2 Results 17
4.3 Discussion 17

5 Robustness under artificially inflated Input **19**
5.1 Setting 19
5.2 Results 19

5.3 Discussion 19

6 Perception-based Experiments **21**
6.1 Setting 21
6.2 Results 21
6.3 Discussion 21

7 Conclusion **23**

A Nomenclature **25**

B Additional Proofs **27**

C Complete Simulation Results **29**

Bibliography **31**

List of Figures

List of Tables

Chapter 1

Introduction

Over the recent years, the field of robotics has seen rapid advancements in the capabilities of neural-network-based implementations. An increase in global compute power[CITE: TODO compute], fueled by the so-called "AI race" of LLMs, has allowed researchers around the world to create even more extensive model architectures for more complex use cases [CITE: TODO llm example]. Transferring these structuresto physical bodies requires more space and on-device compute, often the limiting factors of embodiment. To bridge this gap between powerful models and limited resources, the approach of cheap design has become more common. As mentioned by N. Ay[CITE: TODO ay geometric], the core idea is to maintain an "embodied universal approximator", while reducing the overall complexity of the model, which can be defined based on the specific case. The most straightforward approach is inducing the underlying physical dynamics of the embodiment into the architecture, which is often of lower dimensionality than the actual sensors and actuators. Recent papers focusing on algorithms for the LQR[CITE: TODO lqr], apply this approach by offering implementations for enforcing low-rank controllers, leading to lower complexity in relation to parameterization. Still, this low-dimensional approach has yet to be extended[CITE: TODO extension] to more complex embodiments, and is missing a crucial part required for implementation: Robustness. This leaves us with the question: *Is it possible to train a low-dimensional controller for an embodied agent, that is performant yet robust?*

As robustness is a wide topic, in our case, we use it when referring to sensor- and actuator noise, as well as parametric uncertainties of the underlying model and external disturbances. When talking about performance, we mean reducing cost as much as possible in relation to an optimal controller. In the end, we want to show in which cases low-dimensional controllers are a viable alternative, and can thus reduce costs for hardware implementations.

To answer our question, we will first explain the core concepts and collect existing approaches for the LQR setting in section 2 and 3. In section 4, we will compare these implementations using the double integrator problem for LQR with

respect to general performance, as well as robustness. Building on that, we will artificially inflate our inputs and outputs via matrix multiplication in section 5 and additionally introduce deep RL baseline models into the comparison. For section 6, we will completely move away from LQR, and extend the setting to a perception-based implementation. Here we will compare different models with varying parameter complexity in regard to robustness, applying the concepts we learned from our earlier experiments. Finally, we will conclude by analyzing whether we indeed achieved our goal of constructing a robust controller, and stating how our approach can be applied to more complex settings.

Chapter 2

Preliminaries

Before turning to the experiments, this chapter revisits the concepts that all later parts of this thesis build on. Section 2.1 introduces linear state-space models and their discretization, Section 2.2 shows them on the double integrator that serves as our running example throughout, and Sections 2.3 and 2.4 recall the stability notions and the linear quadratic regulator. Sections 2.5 and 2.6 define the perturbation models and the robustness metrics used to evaluate every controller we construct.

2.1 Linear State-Space Models

In control theory, the evolution of an agent and together with its environment is described by a *state-space model*: a rule that assigns to every point of a state space the instantaneous rate at which the system moves through it. Throughout this thesis we restrict attention to the class of *linear time-invariant* (LTI) systems, for which this rule is linear in both the state and the input and does not itself depend on time[CITE: ControlSystemsTheoryAndDesign.pdf].

A continuous-time LTI system is given by

$$\dot{x}(t) = A_c\, x(t) + B_c\, u(t), \qquad x(0) = x_0, \tag{2.1a}$$

$$y(t) = C\, x(t) + D\, u(t), \tag{2.1b}$$

where $x(t) \in \mathbb{R}^n$ is the *state vector*, $u(t) \in \mathbb{R}^m$ the *input vector* chosen by the agent, and $y(t) \in \mathbb{R}^p$ the *output vector*, i.e. the output observed and thus available to the feedback controller. Accordingly, $A_c \in \mathbb{R}^{n\times n}$ is called the *system matrix*, $B_c \in \mathbb{R}^{n\times m}$ the *input matrix*, $C \in \mathbb{R}^{p\times n}$ the *output matrix*, and $D \in \mathbb{R}^{p\times m}$ the *feedthrough matrix*.

The two equations in (2.1) play fundamentally different roles, and it is helpful explaining their distinction. Equation (2.1a) is a differential equation: it does not

compute the state, which is assumed to be known at every instant, but prescribes its *rate of change*. The pair (A_c, B_c) therefore governs how the system *behaves*, with A_c describing its autonomous dynamics and B_c describing how the input of the agent influences them. Equation (2.1b), in contrast, contains no derivative at all. It is an instantaneous readout, and the pair (C, D) therefore describes how the system is *observed* rather than how it evolves. We set $D = 0$ throughout, which is the standard assumption for systems in which the actuator has no direct instantaneous influence on the sensor[CITE: either same notes or textbook].

Given an input signal $u(\cdot)$, the trajectory of (2.1a) is obtained by integration and is given in closed form by the variation-of-constants formula

$$x(t) = e^{A_c t} x_0 + \int_0^t e^{A_c (t-s)} B_c \, u(s) \, \mathrm{d}s, \tag{2.2}$$

where $e^{A_c t}$ denotes the matrix exponential[CITE: same textbook]. This makes the interpretation of A_c precise: in the scalar case $\dot{x} = ax$ yields $x(t) = e^{at} x_0$, so a is a growth *rate* rather than a value. Equation (2.2) generalises this, with the eigenvalues of A_c acting as growth rates along the corresponding eigendirections.

Full state measurement. Generally, we assume that the state is fully measurable and therefore set $C = I_n$, which reduces (2.1b) to $y(t) = x(t)$. However, when we introduce disturbances through noise, this will no longer be the case. More details on this will be given in Section 2.5. The distinction between $u = -Kx$ and $u = -Ky$ is therefore critical, since the former admits an exact solution while the latter does not; we return to this point in Section 2.4.

2.1.1 Discretization

Digital controllers, as in our case, do not act continuously: they sample the output at a fixed rate and hold the computed controlinput constant until the next sampling point. The learning algorithms studied in this thesis operate on such sampled trajectories as well, so our model has to reflect this. The standard approach is the *zero-order hold* (ZOH)[CITE: find reference]. Fixing a sampling period $h > 0$ and writing $x_k \coloneqq x(kh)$, the input is assumed to be piecewise constant,

$$u(t) = u_k \qquad \text{for } t \in [kh, \, (k+1)h), \quad k \in \mathbb{N}_0. \tag{2.3}$$

Substituting (2.3) into (2.2) over a single sampling interval and applying the change of variables $\tau = (k+1)h - s$ yields the discrete-time model

$$x_{k+1} = A_d \, x_k + B_d \, u_k, \tag{2.4}$$

with

$$A_d = e^{A_c h}, \qquad B_d = \left(\int_0^h e^{A_c \tau} \, \mathrm{d}\tau \right) B_c. \tag{2.5}$$

Two clarifications need to be made. First, (2.4) is not an approximation: it reproduces the continuous-time trajectory exactly at the sampling instants, provided the input is genuinely piecewise constant. This distinguishes ZOH from other schemes such as the forward Euler discretization $A_d \approx I + hA_c$, $B_d \approx hB_c$, which is only first-order accurate.[1] Second, the left-hand side of (2.4) is the actual next state rather than a derivative. The identity $(x_{k+1} - x_k)/h \approx \dot{x}(kh)$ recovers the continuous picture, and the identity block contained in A_d but not in A_c is exactly the state carried over from the previous step.

Unless stated otherwise, all experiments in this thesis are carried out in discrete time with sampling period $h = 0.01$, and we drop the subscript d in the following, writing simply A and B.

2.2 The Double Integrator

To make the comparisons across all chapters meaningful, all experiments are built on a single system: the *double integrator*, a frictionless point mass of mass m moving along a line under an applied force u[CITE: find some reference]. Its dynamics are Newton's second law,

$$m\,\ddot{p}(t) = u(t), \tag{2.6}$$

where $p(t)$ denotes the position of the mass. We simplify by using $m = 1$, so that force and acceleration coincide numerically. Choosing position and velocity as the state, $x = [\,p,\ v\,]^\top \in \mathbb{R}^2$, the second-order equation (2.6) decomposes into the first-order system

$$\dot{p}(t) = v(t), \qquad \dot{v}(t) = u(t), \tag{2.7}$$

in which the first equation is the definition of velocity and the second is (2.6) itself. In the notation of (2.1) this gives

$$A_c = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad C = I_2, \qquad D = 0. \tag{2.8}$$

The entry $(A_c)_{12} = 1$ should be read as stated in Section 2.1: the rate of change of position is given by the current velocity.

[1] For the double integrator of Section 2.2 the two approaches happen to agree on A_d but *not* on B_d, where forward Euler gives $[0,\ h]^\top$ instead of the correct $[h^2/2,\ h]^\top$.

Because $A_c^2 = 0$, the matrix exponential in (2.5) truncates after two terms, $e^{A_c h} = I + A_c h$, and the discretization is available in closed form:

$$A_d = \begin{bmatrix} 1 & h \\ 0 & 1 \end{bmatrix}, \qquad B_d = \begin{bmatrix} h^2/2 \\ h \end{bmatrix}. \tag{2.9}$$

This system is the minimal nontrivial choice for our purposes. It is second-order, so the state is not directly actuated and the controller must account for momentum; it is controllable for every $h > 0$; and it is not open-loop stable, since A_c has a double eigenvalue at the origin and A_d correspondingly a double eigenvalue at 1. The last property matters for the learning experiments: an uncontrolled or badly initialised policy diverges, so the assumption of a known stabilising initial gain (Section 2.4) is a real restriction. These properties are made precise in Section 2.3.

2.3 Characteristics of a system

Now that we have defined our system, before we can optimize over it, it is important to clear up different characteristics of the same. The three we need are properties of the model and not of any particular controller. Stability is what the controller has to establish in the first place, controllability is what guarantees that it can, and observability is what allows a controller that only sees y to act as if it actually recorded x.

2.3.1 Stability

The first of these is stability, which guarantees that deviations in the system don't become too drastic, and thus cause some kind of failure. Mathematically, stability is a statement about a *solution* of the recursion $x_{k+1} = f(x_k, k)$, and is defined the following way[CITE: Astrom & Wittenmark, "Computer-Controlled Systems", p.77]:

Definition 2.1 (Stability)**.** Let $x^0(\cdot)$ be a solution of $x_{k+1} = f(x_k, k)$. The solution $x^0(\cdot)$ is called *stable* if for every $\varepsilon > 0$ there exists a $\delta(\varepsilon, k_0) > 0$ such that every solution $x(\cdot)$ with $\|x(k_0) - x^0(k_0)\| < \delta$ satisfies

$$\|x(k) - x^0(k)\| < \varepsilon \qquad \text{for all } k \geq k_0.$$

It is called *asymptotically stable* if it is stable and in addition $\|x(k) - x^0(k)\| \to 0$ as $k \to \infty$.

In the definition, ε bounds how far a perturbed trajectory is allowed to deviate, and δ says how small the initial distance must be to stay inside that bound. Asymptotic additionally requires the deviation to vanish with increasing time.

For linear systems this simplifies considerably. If $x(\cdot)$ and $x^0(\cdot)$ both solve $x_{k+1} = Ax_k$, then their difference solves the same recursion, so the behaviour of *one* solution decides the behaviour of all of them. Stability is thus a property of the system, explicitly the matrix A. This is stated as a theorem by[CITE: Astrom & Wittenmark, "Computer-Controlled Systems"

Theorem 2.1 (Asymptotic stability of discrete-time LTI systems)**.** *The discrete-time LTI system $x_{k+1} = Ax_k$ is asymptotically stable if and only if all eigenvalues of A lie strictly inside the unit disk, i.e.*

$$\rho(A) \coloneqq \max_i |\lambda_i(A)| < 1 \tag{2.10}$$

Our double integrator sits precisely on the boundary. From (2.9), A_d has the double eigenvalue $\lambda_{1,2} = 1$, so $\rho(A_d) = 1$ and Theorem 2.1 fails. It is not only non-asymptotically stable but actually unstable. We can see that in the matrix powers

$$A_d^k = \begin{bmatrix} 1 & kh \\ 0 & 1 \end{bmatrix}, \tag{2.11}$$

which are growing without bound in k. This is also logical from a physical point of view as a frictionless mass with a nonzero initial velocity continues its trajectory forever if nothing acts on it. Every controller we construct therefore has to *create* stability rather than preserve it.

2.3.2 Controllability

Whether a system is controllable or not indicates if, independently from the controller, it is possible to reach any state inside its state space in a given time. A more formal definition is given in [CITE: ControlSystemsTheoryAndDesign]:

Definition 2.2 (Controllability)**.** The system is said to be *controllable* if for any initial state x_0 and any final state x_f there exist a horizon N and inputs $u_0, \dots, u_{N-1}$ such that the solution of (2.4) satisfies $x_N = x_f$. Otherwise, the system is said to be *uncontrollable*.

To check whether a given system is actually controllable, we need to compute the *controllability matrix*[CITE: ControlSystemsTheoryAndDesign]

$$\mathcal{C}(A, B) = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix} \in \mathbb{R}^{n \times nm}, \tag{2.12}$$

and then verify that it has full rank n; the system is controllable if and only if it does[CITE: ControlSystemsTheoryAndDesign]. Note that (2.12) is stated for a general input matrix B; for a single input ($m = 1$) it reduces to the square matrix $[\, b \;\; Ab \;\; \cdots \;\; A^{n-1}b \,]$.

Applying this to our double integrator, we obtain

$$\mathcal{C}(A_d, B_d) = \begin{bmatrix} B_d & A_dB_d \end{bmatrix} = \begin{bmatrix} h^2/2 & 3h^2/2 \\ h & h \end{bmatrix}, \qquad \det \mathcal{C}(A_d, B_d) = -h^3, \tag{2.13}$$

which is nonzero for every sampling period $h > 0$. The system is therefore controllable at every sampling rate, as claimed in Section 2.2.

For the optimal control problem of Section 2.4 we will not need the full strength of Definition 2.2. The weaker notion of *stabilizability* suffices:

TODO: Stabilizability definition + citation

2.3.3 Observability

Finally, we need to take a look at observability. Above we assumed that we can actually observe the state of the system we are modelling. However, it is much more realistic that we only collect a recording of it, for example through sensors, and need to estimate real state of the system. The notion of observability describes whether we can determine this state from our control input u and our recorded response y alone [CITE: ControlSystemsTheoryAndDesign].

Definition 2.3 (Observability)**.** The system with state-space model (2.4) and output $y_k = Cx_k$ is said to be *observable* if the initial state x_0 can be uniquely determined from the time history of the input u_k and the output y_k over a finite interval. Otherwise, the system is said to be *unobservable*.

Dual to the controllability matrix, we can derive the *observability matrix*

$$\mathcal{O}(C, A) = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} \in \mathbb{R}^{pn \times n}, \tag{2.14}$$

and make use of the following theorem[CITE: ControlSystemsTheoryAndDesign]:

Theorem 2.2 (Observability rank condition)**.** *The system is observable if and only if the observability matrix $\mathcal{O}(C, A)$ has full rank n.*

Using Theorem 2.2 for our double integrator is actually quite simple: we set $C = I_2$ in (2.8), so $\mathcal{O}(C, A_d)$ already contains the identity in its first block rows and has full rank. The system is thus observable.

If we cannot uniquely determine x_k from our observation, we work with an estimate $\hat{x}_k$ instead. Running a copy of the model, $\hat{x}_{k+1} = A\hat{x}_k + Bu_k$, the estimation error $\tilde{x}_k := x_k - \hat{x}_k$ follows the dynamics

$$\tilde{x}_{k+1} = A\,\tilde{x}_k, \qquad \tilde{x}_0 = x_0 - \hat{x}_0, \tag{2.15}$$

which is again just the system without control input. The decay of the error therefore depends only on the eigenvalues of A. If the system is unstable, the error grows infinitely[CITE: ControlSystemsTheoryAndDesign] and for our example it does

so linearly, by (2.11). Observability fixes this by correcting the model with the measurement, $\hat{x}_{k+1} = A\hat{x}_k + Bu_k + L(y_k - C\hat{x}_k)$, which turns (2.15) into $\tilde{x}_{k+1} = (A - LC)\tilde{x}_k$. If and only if the system is observable we can choose L such that the eigenvalues of $A - LC$ lie inside the unit disk, making it stable.

As with stabilizability, the weaker requirement that only the unobservable modes be stable is called *detectability*, and it is the condition we will actually use in Section 2.4.

TODO: Detectability

The topic of observability will become important again in Section 2.5, where we deliberately introduce noise so that $y_k \neq x_k$. This doesn't change our matrix $C = I_2$, so the rank condition of Theorem 2.2 still holds and the system remains observable by definition. However, y_k is no longer a deterministic function of x_k, so we can never estimate the state exactly. We revisit this and its implications there.

2.4 The Linear Quadratic Regulator

Now that we have defined our state-space model and its important characteristics, it is time to introduce our controller, which is based on the *linear quadratic regulator* (LQR) problem.

Here, the setting is that our system starts at an arbitrary initial state in its state space, and the goal of our controller is to return to the zero state as fast as possible. At the same time, the control input should be limited to a realistic quantity, such that it could actually be produced by an actuator. Both requirements can be combined into a single cost function, where Q describes the former, while R associates costs to control input. Over a finite horizon T, this gives the cost[CITE: OptimalAndRobustControl.pdf]

$$V_T(x_0, u) = \int_0^T \left(x(t)^\top Q\, x(t) + u(t)^\top R\, u(t)\right) \mathrm{d}t, \qquad Q = Q^\top \succeq 0, \quad R = R^\top \succ 0. \tag{2.16}$$

We notice to further restrictions here: Q is required to be positive semi-definite, meaning deviating from the origin isn't reducing costs, and R must be positive definite, such that every control input results in a cost.

As the original formula relates to a continuous problem with a fixed endtime T, it doesn't fit well to our goal of evaluating over an infinite discrete horizon. The following formula offers exactly that:

$$J(x_0, u) = \sum_{k=0}^{\infty} \left(x_k^\top Q\, x_k + u_k^\top R\, u_k\right). \tag{2.17}$$

2.4.1 The optimal gain

What makes the LQR problem special is that its solution is a static linear feedback law: the optimal input at time k depends only the current state of the model. Introducing a gain matrix $K \in \mathbb{R}^{m \times n}$, we write it as

$$u_k = -Kx_k. \tag{2.18}$$

TODO - Footnote: Note that the minus in front of the K has been chosen as the convention for this paper. Other formulas have been adapted to this convention, if the other papers used different approaches.

Our controller is thus reduced to choosing a matrix, and the optimal controller is given by the optimal gain matrix K^*.

As we are dealing with a discrete-time system, that gain is obtained from the *discrete algebraic Riccati equation* (DARE)

$$P = A^\top PA - A^\top PB \left(R + B^\top PB\right)^{-1} B^\top PA + Q, \tag{2.19}$$

whose stabilizing solution $P = P^\top \succeq 0$ yields

$$K^* = \left(R + B^\top PB\right)^{-1} B^\top PA, \qquad u_k^* = -K^* x_k. \tag{2.20}$$

The matrix P is additioanlly our *cost-to-go* matrix, meaning that we can use it to calculate the optimal cost incurred from any state x onwards with

$$J^*(x) = x^\top Px. \tag{2.21}$$

Whether (2.19) has such a solution at all is answered by the structural properties of Section 2.3[CITE: OptimalAndRobustControl.pdf]:

Theorem 2.3 (Existence and uniqueness)**.** *Let (A, B) be stabilizable and let $(A, Q^{1/2})$ be detectable. Then the DARE (2.19) has a unique symmetric positive semi-definite solution P, which is the only stabilizing one, and the resulting closed loop satisfies $\rho(A - BK^*) < 1$. If $(A, Q^{1/2})$ is observable, P is positive definite.*

2.4.2 The cost as a function of the gain

Equation (2.20) solves the problem, but it solves it directly and tells us nothing about the quality of non-optimal gains. The learning algorithms of the following chapters never see the model itself; they search over gains directly, and for that we need the cost as a function of K rather than of an input sequence.

Fix a gain K and write the resulting closed loop as $x_{k+1} = A_K x_k$ with $A_K := A - BK$. Substituting (2.18) into (2.17) shows that the cost is again a quadratic

form, $J(x_0, K) = x_0^\top P_K x_0$, where P_K now solves the far simpler discrete *Lyapunov equation*

$$P_K = A_K^\top P_K A_K + Q + K^\top R K, \tag{2.22}$$

which is linear in the unknown P_K and has a unique solution whenever $\rho(A_K) < 1$. This restricts us to the set of stabilizing gains

$$\mathcal{K} \coloneqq \left\{ K \in \mathbb{R}^{m \times n} : \rho(A - BK) < 1 \right\}, \tag{2.23}$$

outside of which we set the cost to $+\infty$, consistent with the remark after (2.17).

One dependency remains, namely on the initial state. Evaluating the gain based on a single x_0 would reward gains that happen to fit that one point, so we instead draw the initial state from a distribution, $x_0 \sim \mathcal{D}$, and average. Writing $\Sigma \coloneqq \mathbb{E}_{x_0 \sim \mathcal{D}}[x_0 x_0^\top]$ for its second moment, the objective becomes

$$f(K) \coloneqq \mathbb{E}_{x_0 \sim \mathcal{D}} \left[x_0^\top P_K x_0 \right] = \operatorname{tr}\left(P_K \Sigma \right), \tag{2.24}$$

using the cyclic property of the trace. Equation (2.24) is the object that every remaining chapter of this thesis minimizes, and the algorithms differ only in how much of the model is known to them.

Throughout this thesis we set $Q = I_2$ and $R = 1$, which weighs position, velocity and control effort equally. Initial states are drawn uniformly from the box $[-10, 10] \times [-1, 1]$.

2.5 Uncertainty and Disturbance Models

Until now, we assumed that we can fully observe the system, and that its behaviour is governed by exactly the dynamics we know. However, in reality there are influences on such systems which we don't clearly know of. Especially for embodied agents this is important, as the world is a messy place.

We will thus introduce four different disturbances to be included in the model, commonly known as noise. They are distinguished by the point at which they affect the loop: what the controller *observes* (measurement noise), what its output actually *does* (control noise), what the plant *is* (parametric uncertainty), and what else *acts* on it (external disturbance). Each of them can be activated separately, so that we can clearly isolate the cause of an effect. Throughout this section k indexes the sampling instants, and σ denotes the standard deviation of the respective noise.

2.5.1 Measurement Noise

This one mainly relates to observability: we influence the sensors used to measured the model state, such that capturing the same state twice would not lead to the same observation y_k. Let n_k^{m} denote the measurement noise at step k. Then

$$n_k^{\mathrm{m}} \sim \mathcal{N}\big(0, \sigma_{\mathrm{m}}^2 I_n\big), \qquad y_k = x_k + n_k^{\mathrm{m}}, \qquad \sigma_{\mathrm{m}} \in \mathbb{R}_+, \tag{2.25}$$

drawn independently at every step.

While one might think that this noise removes observability for us, it doesn't. The output matrix is still $C = I_2$, so the rank condition of Theorem 2.2 holds unchanged and observability is given.

However, it does remove the deterministic link between state and output, so we can't identify x_k exactly with any amount of measurements. Using our noisy observation in the feedback law (2.18) and $u_k = -Ky_k$, the closed loop becomes

$$x_{k+1} = (A - BK)\, x_k - BK\, n_k^{\mathrm{m}}. \tag{2.26}$$

So even a perfectly stabilizing K can no longer bring the state exactly to the origin, but rather to a distribution around it.

2.5.2 Control Noise

Control noise, as the name says, acts on the applied control input. With it, we try to model thedifference between the intended movement of an actuator, and its actual action due to different factors. Here we deal with *multiplicative* noise, which scales with the input. In detail this means that the applied control input v_k is obtained as

$$n_k^{\mathrm{c}} \sim \mathcal{N}\big(0, \sigma_{\mathrm{c}}^2 I_m\big), \qquad v_k = (1 + n_k^{\mathrm{c}}) \odot u_k, \tag{2.27}$$

where $\odot$ denotes the elementwise product, so that σ_{c} is the *relative* error. Applying this to the input, the next step becomes

$$x_{k+1} = A\, x_k + B\, v_k. \tag{2.28}$$

2.5.3 Parametric Uncertainty

This noise acts directly on the matrices of the model. Here, we either decide to keep it time invariant, meaning $\Delta_k = \Delta_0$ for all k, or to resample it at every step. The former models a wrongly identified system, while the latter simulates one that continuously changes. As with measurement noise, it is additive. Applied, it acts on the system the following way:

$$\left(\Delta_k^A\right)_{ij},\ \left(\Delta_k^B\right)_{ij} \overset{\text{i.i.d.}}{\sim} \mathcal{N}\left(0, \sigma_{\text{p}}^2\right), \qquad x_{k+1} = \left(A + \Delta_k^A\right) x_k + \left(B + \Delta_k^B\right) v_k. \tag{2.29}$$

Worth noting is that resampling every step leaves the model correct *on average*, since $\mathbb{E}[\Delta_k] = 0$, and mainly adds variance to the trajectory. This stands in contrast to the time invariant version, where we have an discrepancy between what we compute actions on, and what actually happens.

2.5.4 External Disturbance

This noise models external influences, like a constant wind pushing on our agent, or periodic disturbances, for example rain hitting it with some frequency. Depending on the type, the disturbance $n_k^{\text{d}} \in \mathbb{R}^n$ is drawn as

$$\begin{aligned}
&\text{Gaussian:} && n_k^{\text{d}} \sim \mathcal{N}\left(0, \sigma_{\text{d}}^2 I_n\right) \quad \text{independently at every step,} && \text{(2.30a)}\\
&\text{Sinusoidal:} && n_k^{\text{d}} = \sigma_{\text{d}} \sin(0.1\, k)\, \mathbf{1}_n, && \text{(2.30b)}\\
&\text{Constant:} && n_k^{\text{d}} = n_0^{\text{d}} \sim \mathcal{N}\left(0, \sigma_{\text{d}}^2 I_n\right) \quad \text{for all } k. && \text{(2.30c)}
\end{aligned}$$

Only the first one can really be considered as noise; the other are more deterministic values, where σ_{d} acts as the amplitude of it. The sinusoidal variant completes one period every $2\pi/0.1 \approx 63$ steps, which at a sampling intervall of $h = 0.01$ is a slow drift rather than a rapid vibration. Including the noise in our loop, the state transition becomes

$$x_{k+1} = A\, x_k + B\, v_k + n_k^{\text{d}}. \tag{2.31}$$

2.5.5 The full loop

While we, until now, have only regarded each noise type on their own, there is also the possibility to include them all at once. Doing so, a singular step becomes:

$$\begin{aligned}
y_k &= x_k + n_k^{\text{m}}, && \text{(2.32a)}\\
u_k &= -K y_k, && \text{(2.32b)}\\
v_k &= (1 + n_k^{\text{c}}) \odot u_k, && \text{(2.32c)}\\
x_{k+1} &= \left(A + \Delta_k^A\right) x_k + \left(B + \Delta_k^B\right) v_k + n_k^{\text{d}}. && \text{(2.32d)}
\end{aligned}$$

Relevant to note about (2.32) is that the controller only interacts with y_k and u_k, whereas the dynamics and the stage cost of (2.17) are evaluated on the true state x_k. Thus our controller can never actually observe the true state it is evaluated

on. For our baseline LQR solution of Section 2.4, which we will use to compare, we compute the gain on a pure system, and only afterwards apply it to an *exposed* system of the style (2.32) afterwards. Measuring how much performance this difference makes is the subject of the next section.

2.6 Robustness Metrics

TODO: How do I define robustness, what are interesting metrics to report, per section

2.7 Notation

TODO: Which letters I will use in what context, tbd

Chapter 3

Background

3.1 Motivation

Reviewing content of past papers, state of the art, and what the open problems are.

3.2 Approach

What I will do in the following sections

Chapter 4

Robustness of LQR with low-dimensional Input

4.1 Setting

4.2 Results

4.3 Discussion

Chapter 5

Robustness under artificially inflated Input

5.1 Setting

5.2 Results

5.3 Discussion

Chapter 6

Perception-based Experiments

6.1 Setting

6.2 Results

6.3 Discussion

Chapter 7

Conclusion

You generally cover three things in the Conclusions section, and each of these usually merits a separate subsection:

1. Conclusions
2. Summary of Contributions
3. Future Research

Conclusions are not a rambling summary of the thesis: they are short, concise statements of the inferences that you have made because of your work. It helps to organize these as short numbered paragraphs, ordered from most to least important. All conclusions should be directly related to the research question stated in the introduction. Examples:

1. The problem stated in the introduction has been solved: as shown in Sections ? to ??, an algorithm capable of handling large-scale Zylon problems in reasonable time has been developed.
2. The principal mechanism needed in the improved Zylon algorithm is the Grooty mechanism.
3. Etc.

The Summary of Contributions will be much sought and carefully read by the examiners. Here you list the contributions of new knowledge that your thesis makes. Of course, the thesis itself must substantiate any claims made here. There is often some overlap with the Conclusions, but that's okay. Concise numbered paragraphs are again best. Organize from most to least important. Examples:

1. Developed a much quicker algorithm for large-scale Zylon problems.
2. Demonstrated the first use of the Grooty mechanism for Zylon calculations.
3. Etc.

The Future Research subsection is included so that researchers picking up this work in future have the benefit of the ideas that you generated while you were working on the project. Again, concise numbered paragraphs are usually best.

Appendix A

Nomenclature

What goes in the appendices? Any material which impedes the smooth development of your presentation, but which is important to justify the results of a thesis. Generally it is material that is of too nitty-gritty a level of detail for inclusion in the main body of the thesis, but which should be available for perusal by the examiners to convince them sufficiently. Examples include program listings, immense tables of data, lengthy mathematical proofs or derivations, etc.

Appendix B

Additional Proofs

Appendix C

Complete Simulation Results

Bibliography

Erklärung der Urheberschaft

Ich versichere an Eides statt, dass ich die Bachelor's Thesis im Studiengang Data Science selbstständig verfasst und keine anderen als die angegebenen Hilfsmittel – insbesondere keine im Quellenverzeichnis nicht benannten Internet-Quellen – benutzt habe. Es wurden keine Quellen genutzt, die nicht genannt wurden. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen wurden, sind als solche kenntlich gemacht. Ich versichere weiterhin, dass ich die Arbeit vorher nicht in einem anderen Prüfungsverfahren eingereicht habe und die eingereichte schriftliche Fassung der auf dem elektronischen Speichermedium entspricht.

Ort, Datum Unterschrift

Erklärung zur Veröffentlichung

Ich erkläre mein Einverständnis mit der Einstellung dieser Bachelor's Thesis in den Bestand der Bibliothek.

Ort, Datum Unterschrift